```
// BootRunner.cs
using System.Collections;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using Logger = Sy.Core.Logger;
namespace Sy.P05.Boot
{
    public static class SceneIds
    {
        public const string Boot = "S00_Boot";
        public const string Main = "S05_Main";
    }
    /// &lt;summary&gt;
    /// S00_Boot       Mock  →   →     T001④
    ///      Boot            UI T001b  Boot      =
    /// &lt;/summary&gt;
    public sealed class BootRunner : MonoBehaviour
    {
        IEnumerator Start()
        {
            BootServices.RegisterAll();
            var sequence = new BootSequence(SwitchToMain);
            if (!sequence.RunAll())
            {
                Logger.Error("BOOT", "boot sequence failed - stay on S00_Boot");
                yield break;
            }
            yield return null;
        }
        bool SwitchToMain()
        {
            StartCoroutine(LoadMainRoutine());
            return true;
        }
        IEnumerator LoadMainRoutine()
        {
            var op = SceneManager.LoadSceneAsync(SceneIds.Main, LoadSceneMode.Single);
            while (op != null &amp;&amp; !op.isDone) yield return null;
        }
    }
}
// BootSequence.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
namespace Sy.P05.Boot
{
    /// &lt;summary&gt;
```

```
/// Boot     T001
/// ①Logger→②  →③Config→④Save→⑤ILocale→⑥Ads.Preload→⑦         [BOOT] n/7
///  ⑦        Play  =SceneManager.LoadSceneAsync Editor dry-run=EditorSceneManager.OpenScene
/// X006      SY_MOCK
/// &lt;/summary&gt;
public sealed class BootSequence
{
    public const int TotalSteps = 7;
    readonly Func&lt;bool&gt; _switchScene;
    readonly List&lt;string&gt; _lines = new();
    public BootSequence(Func&lt;bool&gt; switchScene)
    {
        _switchScene = switchScene;
    }
    public IReadOnlyList&lt;string&gt; Lines =&gt; _lines;
    public bool RunAll()
    {
        bool ok = true;
        ok &amp;= Run(1, "Logger", LoggerStep);
        ok &amp;= Run(2, "Time", TimeStep);
        ok &amp;= Run(3, "Config", ConfigStep);
        ok &amp;= Run(4, "Save", SaveStep);
        ok &amp;= Run(5, "ILocale", LocaleStep);
        ok &amp;= Run(6, "Ads.Preload", AdsStep);
        ok &amp;= Run(7, "SwitchMain", () =&gt; _switchScene != null &amp;&amp; _switchScene());
        return ok;
    }
    bool Run(int n, string name, Func&lt;bool&gt; step)
    {
        bool result;
        try
        {
            result = step();
        }
        catch (Exception e)
        {
            Logger.Error("BOOT", $"step {n} {name} exception: {e.Message}");
            result = false;
        }
        string line = $"[BOOT] {n}/{TotalSteps} {name} {(result ? "ok" : "FAIL")}";
        _lines.Add(line);
        Logger.Info("BOOT", line);
        return result;
    }
    static bool LoggerStep() =&gt; true; //              JS sink platform T001c
    static bool TimeStep()
    {
        var time = ServiceLocator.Get&lt;ITimeService&gt;();
        return time.ServerNowMs() &gt; 0 &amp;&amp; !time.TimeCheated();
    }
```

```
        static bool ConfigStep()
        {
            var config = ServiceLocator.Get&lt;IConfigService&gt;();
            var update = config.CheckUpdate("mock://manifest");
            return update.NewVersion != null; // mock=    X009   manifest
        }
        static bool SaveStep()
        {
            var save = ServiceLocator.Get&lt;ISaveService&gt;();
            save.Save("boot_probe", "{}");
            return !string.IsNullOrEmpty(save.Load("boot_probe"));
        }
        static bool LocaleStep()
        {
            var locale = ServiceLocator.Get&lt;ILocaleService&gt;();
            return !string.IsNullOrEmpty(locale.GetFontSet().NormalFont); //         +
        }
        static bool AdsStep()
        {
            if (!AdHubBoot.Wire()) return false;   // P05 AdHub slot table registration (O-20261010-1325 five-slot rollo
            ServiceLocator.Get&lt;IAdsService&gt;().Preload("boot_warmup");
            return true;
        }
    }
}
// BootServices.cs
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Core;
using Sy.Platform;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.P05.Boot
{
    /// &lt;summary&gt;
    ///    Mock    SY_MOCK    X006     Mock X002a   AppID touristappid
    /// T001c  WX/TT     Channel   Editor      Boot
    /// &lt;/summary&gt;
    public static class BootServices
    {
        public static void RegisterAll()
        {
            if (ServiceLocator.TryGet&lt;ITimeService&gt;(out _)) return; //
            ServiceLocator.Register&lt;ITimeService&gt;(new MockTimeService());
            ServiceLocator.Register&lt;IConfigService&gt;(new MockConfigService());
            ServiceLocator.Register&lt;ISaveService&gt;(new MockSaveService());
            ServiceLocator.Register&lt;ILocaleService&gt;(new MockLocaleService());
            ServiceLocator.Register&lt;IAdsService&gt;(new MockAdsService());
            Logger.Info("BOOT", "mock services registered (SY_MOCK)");
```

```
    }
}
public sealed class MockTimeService : ITimeService
{
    public TimeSource Source =&gt; TimeSource.Mock;
    public long ClientNowMs() =&gt; DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
    /// &lt;summary&gt;SY_MOCK X006                  TCB serverTime→HTTPS Date →   T001b   &lt;/summary&gt;
    public long ServerNowMs() =&gt; ClientNowMs();
    public bool TimeCheated() =&gt; Math.Abs(ServerNowMs() - ClientNowMs()) &gt; 120_000;
}
public sealed class MockConfigService : IConfigService
{
    public T GetTable&lt;T&gt;(string tableId) where T : class
        =&gt; throw new KeyNotFoundException("mock config table not loaded: " + tableId);
    public bool TryGetTable&lt;T&gt;(string tableId, out T table) where T : class
    {
        table = null;
        return false;
    }
    public ConfigUpdateResult CheckUpdate(string manifestUrl)
        =&gt; new ConfigUpdateResult(false, "mock-0.0.1");
    public void SwapPolicy(ConfigSwapPolicy policy) { }
    public string PinConfigVersion(string scope) =&gt; "mock-0.0.1";
}
public sealed class MockSaveService : ISaveService
{
    const string Prefix = "sy_save_";
    public void Save(string slot, string payloadJson) =&gt; PlayerPrefs.SetString(Prefix + slot, payloadJson);
    public string Load(string slot) =&gt; PlayerPrefs.GetString(Prefix + slot, string.Empty);
    /// &lt;summary&gt;X006             T001b v1→v2     &lt;/summary&gt;
    public SyncResult CloudSync() =&gt; new SyncResult(false, null);
}
public sealed class MockLocaleService : ILocaleService
{
    public void SetLang(string lang) { }
    /// &lt;summary&gt; key  key      T001b cfg_i18n +  NUnit    &lt;/summary&gt;
    public string GetText(string key, params string[] args)
    {
        Logger.Warn("LOCALE", "missing i18n key (fallback): " + key);
        return key;
    }
    public FontSet GetFontSet()
        =&gt; new FontSet("LegacyRuntime.ttf", "LegacyRuntime.ttf", "LegacyRuntime.ttf");
}
public sealed class MockAdsService : IAdsService
{
    public event Action&lt;AdLifecycle&gt; OnAdEvent;
    public Task&lt;RewardResult&gt; ShowRewarded(string pointId)
        =&gt; Task.FromResult(new RewardResult { Completed = true, Reason = "mock" });
    public void ShowInterstitial(string pointId) { }
```

```
        public void ShowBanner(string pointId) { }
        public void Preload(string pointId)
        {
            Logger.Info("ADS", "preload (mock): " + pointId);
            OnAdEvent?.Invoke(new AdLifecycle(AdLifecycleType.Fill, pointId, "mock"));
        }
    }
}
// MainUI.cs
using UnityEngine;
using UnityEngine.SceneManagement;
namespace Sy.P05.Newbie
{
    public static class NewbieSceneIds
    {
        public const string Main = "S05_Main";
        public const string Battle = "S05_Battle";
    }
    /// &lt;summary&gt;S05          →S05_Battle M5      ——           &lt;/summary&gt;
    public sealed class MainUI : MonoBehaviour
    {
        public void StartSurvival()
        {
            SceneManager.LoadScene(NewbieSceneIds.Battle);
        }
        void Awake()
        {
            Application.targetFrameRate = 30;
            Sy.Core.MainArtInjector.Apply(transform, "P05_scene_01", "P05_general_01_zhaoyun_awake");
        }
    }
}
// AdHubBoot.cs
using Sy.AdHub;
namespace Sy.P05.Boot
{
    /// &lt;summary&gt;
    /// P05 slot table for the shared AdHub layer (O-20261010-1325 rollout, design case five slots).
    /// Tuning numbers live in the JSON table below (data, not logic). Idle-game frequency band
    /// per design case: daily cap 5-8 plays, cooldown 60-90s. Every reward face stays &lt;= 2x gain.
    /// &lt;/summary&gt;
    public static class AdHubBoot
    {
        // Design case slots: offline_x2 / dig_boost / cold_relief / build_fast / hero_extra.
        const string SlotTableJson =
            "{\"slots\":[" +
            "{\"slot\":\"offline_x2\",\"rewardFace\":\"offline furnace income x2\",\"dailyCap\":5,\"cooldownSec\":75,\"i
            "{\"slot\":\"dig_boost\",\"rewardFace\":\"hero dig instant finish, dig yield x2\",\"dailyCap\":6,\"cooldownS
            "{\"slot\":\"cold_relief\",\"rewardFace\":\"temp anti-cold fuel pack\",\"dailyCap\":6,\"cooldownSec\":75,\"i
            "{\"slot\":\"build_fast\",\"rewardFace\":\"finish one camp build queue instantly\",\"dailyCap\":6,\"cooldown
```

```
            "{\"slot\":\"hero_extra\",\"rewardFace\":\"one extra hero recruit draw\",\"dailyCap\":5,\"cooldownSec\":75,\
        public const int SlotCount = 5;
        public static AdSlotTable Table =&gt; AdSlotTable.FromJson(SlotTableJson);
        /// &lt;summary&gt;Boot step 6 hook: registers game id and slot table. False only on a broken table.&lt;/summary
        public static bool Wire()
        {
            var table = Table;
            if (table.slots == null || table.slots.Length != SlotCount) return false;
            // Fully qualified: inside namespace Sy.P05.*, the sibling namespace Sy.AdHub shadows
            // the using-imported class AdHub during simple-name lookup (CS0234).
            Sy.AdHub.AdHub.SetGameId("P05");
            Sy.AdHub.AdHub.SetSlotTable(table);
            return true;
        }
    }
}
// BattleLauncher.cs
using System.Collections.Generic;
using UnityEngine;
namespace Sy.P05.Newbie
{
    /// &lt;summary&gt;
    /// S05_Battle       FurnaceSim+TutorialFlow+FurnaceView   Tick
    ///    Sy.Telemetry Editor =Debug sink T001e        +TCB
    /// HUD                     6
    /// &lt;/summary&gt;
    public sealed class BattleLauncher : MonoBehaviour
    {
        FurnaceSim _sim;
        TutorialFlow _tutorial;
        FurnaceView _view;
        Sy.Telemetry.TelemetryService _telemetry;
        int _lastCoins = -1, _lastCitizens = -1, _lastWood = -1, _lastFuel = -1;
        void Awake()
        {
            Application.targetFrameRate = 30;
            var canvas = FindObjectOfType&lt;Canvas&gt;();
            _sim = new FurnaceSim();
            _view = gameObject.AddComponent&lt;FurnaceView&gt;();
            _view.Initialize(canvas, _sim);
            _telemetry = new Sy.Telemetry.TelemetryService();
            _telemetry.AddSink(new TelemetryDebugSink());
            _tutorial = new TutorialFlow(
                _sim,
                hint =&gt;
                {
                    Debug.Log("[HINT] " + hint);
                    _view.SetHint(hint);
                },
                step =&gt; _telemetry.Track(Sy.Telemetry.EventIds.NewbieStep,
```

```
            new Dictionary&lt;string, string&gt; { { "step", step.ToString() } }));
        _view.ClaimRequested += OnClaimRequested;
        _view.FirewoodRequested += OnFirewoodRequested;
        _view.BuildRequested += OnBuildRequested;
        _view.DigRequested += OnDigRequested;
        _view.NearDispatchRequested += () =&gt; OnDispatchRequested(0);
        _view.FarDispatchRequested += () =&gt; OnDispatchRequested(1);
        _view.SocialTapped += OnSocialTapped;
        _sim.PhaseChanged += OnPhaseChanged;
        _sim.DigStageReached += OnDigStageReached;
    }
    void Update()
    {
        _sim.Tick(Time.deltaTime);
        _view.SyncView(_sim);
        if (_sim.Coins != _lastCoins) { _lastCoins = _sim.Coins; _view.SetCoins(_lastCoins); }
        if (_sim.Citizens != _lastCitizens) { _lastCitizens = _sim.Citizens; _view.SetCitizens(_lastCitizens); }
        if (_sim.Wood != _lastWood) { _lastWood = _sim.Wood; _view.SetWood(_lastWood); }
        if ((int)_sim.Fuel != _lastFuel) { _lastFuel = (int)_sim.Fuel; _view.SetFuel(_lastFuel); }
    }
    void OnClaimRequested()
    {
        _sim.ClaimOffline();
    }
    void OnFirewoodRequested()
    {
        _sim.AddFirewood();
    }
    void OnBuildRequested()
    {
        _sim.BuildWoodyard();
    }
    void OnDigRequested()
    {
        _sim.StartDig();
    }
    void OnDispatchRequested(int node)
    {
        _sim.DispatchOrder(node);
    }
    void OnSocialTapped()
    {
        _view.SetHint("          ——              1 / ·      ");
    }
    void OnDigStageReached(int stage)
    {
        _view.SetDigStage(stage);
    }
    void OnPhaseChanged(FurnaceSim.SimPhase phase)
    {
```

```
            if (phase == FurnaceSim.SimPhase.Settled)
                _view.ShowSettle(_sim.Coins, _sim.Citizens, _sim.Wood, (int)_sim.Fuel, _sim.Day, _sim.OutsideDispatchDon
            else
                _view.HideIntro();
        }
        sealed class TelemetryDebugSink : Sy.Telemetry.TelemetryService.ISink
        {
            public void Write(string evtId, Dictionary&lt;string, string&gt; fields)
            {
                var sb = new System.Text.StringBuilder(evtId);
                foreach (var kv in fields)
                {
                    sb.Append(' ').Append(kv.Key).Append('=').Append(kv.Value);
                }
                Debug.Log("[TELEMETRY] " + sb);
            }
        }
    }
}
// FurnaceSim.cs
using System;
namespace Sy.P05.Newbie
{
    /// &lt;summary&gt;
    ///              C#       Play   Editor headless   NewbieFlowCheck
    ///          §4        ticking  (fuelPerMin=4+fireLv*2·     )/  lv1  /    →    /
    ///         (    ×2+frostRisk+      )/        /
    ///      06  05   M0       →M1   (8h  )+    (  1 ·  +30)→M2    lv1
    /// (    +   lv2·    )→M3    (10s   )    →M4     +      (    /       )
    /// →M5     +      +  "    "(    +1)
    ///                               1        =
    /// &lt;/summary&gt;
    public sealed class FurnaceSim
    {
        public enum SimPhase
        {
            Intro, AwaitClaim, AwaitFirewood, AwaitBuild, AwaitDig, Digging,
            AwaitDispatch, Dispatching, Reveal, Settled
        }
        public SimPhase Phase { get; private set; } = SimPhase.Intro;
        public float Fuel { get; private set; } = NewbieConfig.StartFuel;
        public int Coins { get; private set; }
        public int Citizens { get; private set; } = NewbieConfig.StartCitizens;
        public int Wood { get; private set; } = NewbieConfig.StartWood;
        public int Day { get; private set; } = NewbieConfig.StartDay;
        public int FireLevel { get; private set; } = 1;               //         §4.2①   →
        public bool HasWoodyard { get; private set; }
        public bool SurvivorFound { get; private set; }
        public float PhaseTimer { get; private set; }
        public float LightRadius =&gt; NewbieConfig.LightRadiusByLevel[FireLevel];
```

```
public int DigStagesSeen { get; private set; }              // bit0-3=
public bool OutsideDispatchDone { get; private set; }        //          ×2+  +
public int AdPopupCount =&gt; 0;                                 //
public event Action&lt;SimPhase&gt; PhaseChanged;
public event Action Claimed;
public event Action FirewoodAdded;
public event Action WoodyardBuilt;
public event Action FireLevelUp;                             //          =M5
public event Action DigStarted;
public event Action&lt;int&gt; DigStageReached;                     //            V1.2  ②
public event Action SurvivorDug;
public event Action&lt;int&gt; DispatchStarted;                     // (    0  /1  )
public event Action&lt;int, int&gt; OrderDone;                     // (    ,    )
public event Action FrostRiskWarned;                         //        frostRisk
public event Action Settled;
int _dispatchNode;
int _digStageDone;
float _woodTimer;
public void Tick(float dt)
{
    if (Phase == SimPhase.Intro)
    {
        PhaseTimer += dt;
        if (PhaseTimer &gt;= NewbieConfig.IntroSeconds) SetPhase(SimPhase.AwaitClaim);
        return;
    }
    if (Phase == SimPhase.Settled) return;
    //   ticking    §4.1  fuelPerMin=4+fireLv*2          1
    Fuel = Math.Max(NewbieConfig.FuelFloor, Fuel - NewbieConfig.FuelDrainPerSecond(FireLevel) * dt);
    //   lv1         WoodyardWoodInterval +1    WoodyardWoodCap
    if (HasWoodyard)
    {
        _woodTimer += dt;
        if (_woodTimer &gt;= NewbieConfig.WoodyardWoodInterval &amp;&amp; Wood &lt; NewbieConfig.WoodyardWoodCap
        {
            _woodTimer = 0f;
            Wood++;
        }
    }
    switch (Phase)
    {
        case SimPhase.Digging:
            PhaseTimer += dt;
            TickDigStages();
            if (PhaseTimer &gt;= NewbieConfig.DigSeconds)
            {
                SurvivorFound = true;
                Citizens++;
                SurvivorDug?.Invoke();
                SetPhase(SimPhase.AwaitDispatch);
```

```
                }
                break;
            case SimPhase.Dispatching:
                PhaseTimer += dt;
                if (PhaseTimer &gt;= NewbieConfig.DispatchSeconds)
                {
                    Wood += NewbieConfig.DispatchYield[_dispatchNode];
                    if (_dispatchNode == 1)
                    {
                        OutsideDispatchDone = true;
                        Fuel = Math.Max(NewbieConfig.FuelFloor, Fuel - NewbieConfig.OutsideWarmupFuel);   //
                    }
                    OrderDone?.Invoke(NewbieConfig.DispatchYield[_dispatchNode], _dispatchNode);
                    SetPhase(SimPhase.Reveal);
                }
                break;
            case SimPhase.Reveal:
                PhaseTimer += dt;
                if (PhaseTimer &gt;= NewbieConfig.RevealSeconds)
                {
                    Day++;   //     →    +1    §4.1     +
                    Settled?.Invoke();
                    SetPhase(SimPhase.Settled);
                }
                break;
        }
    }
    void TickDigStages()
    {
        int stage = Math.Min(NewbieConfig.DigStages.Length,
            1 + (int)(PhaseTimer / (NewbieConfig.DigSeconds / NewbieConfig.DigStages.Length)));
        if (stage &gt; _digStageDone)
        {
            _digStageDone = stage;
            DigStagesSeen |= 1 &lt;&lt; (stage - 1);
            DigStageReached?.Invoke(stage);
        }
    }
    /// &lt;summary&gt;M1a      8h     &lt;/summary&gt;
    public bool ClaimOffline()
    {
        if (Phase != SimPhase.AwaitClaim) return false;
        Coins += NewbieConfig.OfflineCoins;
        Claimed?.Invoke();
        SetPhase(SimPhase.AwaitFirewood);
        return true;
    }
    /// &lt;summary&gt;M1b       1 →  +30     &lt;/summary&gt;
    public bool AddFirewood()
    {
```

```
    if (Phase != SimPhase.AwaitFirewood) return false;
    if (Wood &lt; 1) return false;   //        3 ——
    Wood--;
    Fuel = Math.Min(100f, Fuel + NewbieConfig.FirewoodFuel);
    FirewoodAdded?.Invoke();
    SetPhase(SimPhase.AwaitBuild);
    return true;
}
/// &lt;summary&gt;M2    lv1     +  +   lv2     ·     ①  &lt;/summary&gt;
public bool BuildWoodyard()
{
    if (Phase != SimPhase.AwaitBuild) return false;
    HasWoodyard = true;
    FireLevel++;
    FireLevelUp?.Invoke();
    WoodyardBuilt?.Invoke();
    SetPhase(SimPhase.AwaitDig);
    return true;
}
/// &lt;summary&gt;M3                  →10s      &lt;/summary&gt;
public bool StartDig()
{
    if (Phase != SimPhase.AwaitDig) return false;
    PhaseTimer = 0f;
    _digStageDone = 0;
    DigStarted?.Invoke();
    SetPhase(SimPhase.Digging);
    return true;
}
/// &lt;summary&gt;M4              0=    /1=    ·  ×2·frostRisk+        &lt;/summary&gt;
public bool DispatchOrder(int node)
{
    if (Phase != SimPhase.AwaitDispatch) return false;
    if (node &lt; 0 || node &gt;= NewbieConfig.DispatchNames.Length) return false;
    _dispatchNode = node;
    if (node == 1)
    {
        FrostRiskWarned?.Invoke();   // frostRisk=0.05*        15% ——
    }
    PhaseTimer = 0f;
    DispatchStarted?.Invoke(node);
    SetPhase(SimPhase.Dispatching);
    return true;
}
void SetPhase(SimPhase p)
{
    if (Phase == p) return;
    Phase = p;
    PhaseTimer = 0f;
    PhaseChanged?.Invoke(p);
```

```
        }
    }
}
// FurnaceView.cs
using System;
using System.Collections;
using UnityEngine;
using UnityEngine.UI;
namespace Sy.P05.Newbie
{
    /// &lt;summary&gt;
    ///                UGUI——  ×  ×   ×   ×
    ///   UI Initialize          YAML      =    /    /            →  ·     ① /
    ///     vs                   /         /  "    "
    ///                  =
    /// &lt;/summary&gt;
    public sealed class FurnaceView : MonoBehaviour
    {
        Canvas _canvas;
        Sprite _square;
        FurnaceSim _sim;
        RectTransform _lightCircle, _furnaceRt;
        Image _fuelFill, _digFill, _dispatchFill;
        Image _digPortrait;   // X027: dig-stage general art (frozen peek -&gt; awake reveal)
        Button _firewoodButton, _buildButton, _iceButton, _nearButton, _farButton, _socialButton;
        Image _firewoodImage, _buildImage, _iceImage, _nearImage, _farImage;
        Text _coinText, _citizenText, _woodText, _fuelText, _hintText, _digStageText;
        RectTransform _offlinePopup, _settlePanel;
        Text _settleStats;
        FurnaceSim.SimPhase _lastPhase;
        float _circleScale = NewbieConfig.LightRadiusByLevel[1];
        public event Action ClaimRequested;
        public event Action FirewoodRequested;
        public event Action BuildRequested;
        public event Action DigRequested;
        public event Action NearDispatchRequested;   //   ·
        public event Action FarDispatchRequested;    //   ·     ×2·
        public event Action SocialTapped;
        static readonly Color SnowWhite = new Color(0.969f, 0.976f, 0.988f);     // #F7F9FC
        static readonly Color IceBlue = new Color(0.659f, 0.773f, 0.855f);       // #A8C5DA
        static readonly Color InkBlue = new Color(0.118f, 0.227f, 0.373f);       // #1E3A5F
        static readonly Color FurnaceOrange = new Color(1f, 0.42f, 0.102f);     // #FF6B1A
        static readonly Color IceGray = new Color(0.42f, 0.48f, 0.549f);          // #6B7B8C
        public void Initialize(Canvas canvas, FurnaceSim sim)
        {
            _canvas = canvas;
            _sim = sim;
            _square = MakeSquareSprite();
            BuildScene();
            BuildHud();
```

```
        BuildOfflinePopup();
        BuildSettlePanel();
        SetHint("        ……");
        StartCoroutine(SnowRoutine());
        StartCoroutine(FlameRoutine());
    }
    // ————    ————
    void BuildScene()
    {
        var bg = MakeFullImage("Bg", InkBlue);
        bg.SetAsFirstSibling();
        // X027 generated art: swap the solid Bg block for the scene painting
        // when present; missing art keeps the procedural look untouched.
        var bgImg = bg.GetComponent&lt;Image&gt;();
        var bgArt = Sy.Core.ArtRuntime.Get("P05_scene_01");
        if (bgArt != null)
        {
            bgImg.sprite = bgArt;
            bgImg.color = Color.white;
            bgImg.preserveAspect = true;
            bgImg.raycastTarget = false;
        }
        //
        var ground = MakeCanvasChild("Ground");
        ground.sizeDelta = new Vector2(1080, 520);
        ground.anchoredPosition = new Vector2(0, -700);
        var groundImg = ground.GetComponent&lt;Image&gt;();
        groundImg.sprite = _square;
        groundImg.color = new Color(0.85f, 0.88f, 0.92f);
        //        →   lv1 0.55 → lv2 0.95
        _lightCircle = MakeCanvasChild("LightCircle");
        _lightCircle.sizeDelta = new Vector2(700, 700);
        _lightCircle.anchoredPosition = new Vector2(0, -420);
        var circleImg = _lightCircle.GetComponent&lt;Image&gt;();
        circleImg.sprite = _square;
        circleImg.color = new Color(IceBlue.r, IceBlue.g, IceBlue.b, 0.10f);
        // X027 generated art: hidden general portrait shown by dig stage below.
        var portraitGo = new GameObject("DigPortrait", typeof(RectTransform), typeof(Image));
        portraitGo.transform.SetParent(_canvas.transform, false);
        _digPortrait = portraitGo.GetComponent&lt;Image&gt;();
        var portraitRt = (RectTransform)portraitGo.transform;
        portraitRt.anchorMin = portraitRt.anchorMax = new Vector2(0.5f, 0.5f);
        portraitRt.anchoredPosition = new Vector2(0, 40);
        portraitRt.sizeDelta = new Vector2(640, 860);
        _digPortrait.raycastTarget = false;
        _digPortrait.gameObject.SetActive(false);
        //
        _furnaceRt = MakeCanvasChild("Furnace");
        _furnaceRt.anchoredPosition = new Vector2(0, -420);
        _furnaceRt.sizeDelta = new Vector2(150, 150);
```

```
var furnaceImg = _furnaceRt.GetComponent&lt;Image&gt;();
furnaceImg.sprite = _square;
furnaceImg.color = FurnaceOrange;
MakeText(_furnaceRt, " ", 60, Vector2.zero, TextAnchor.MiddleCenter, 150, 150).color = InkBlue;
var fuelBar = MakeCanvasChild("FuelBar");
fuelBar.anchoredPosition = new Vector2(0, -560);
fuelBar.sizeDelta = new Vector2(300, 24);
var fuelBg = fuelBar.GetComponent&lt;Image&gt;();
fuelBg.sprite = _square;
fuelBg.color = new Color(0.15f, 0.18f, 0.22f);
var fillGo = new GameObject("Fill", typeof(RectTransform), typeof(Image));
fillGo.transform.SetParent(fuelBar, false);
var fillRt = (RectTransform)fillGo.transform;
fillRt.anchorMin = new Vector2(0f, 0.5f);
fillRt.anchorMax = new Vector2(1f, 0.5f);
fillRt.anchoredPosition = Vector2.zero;
fillRt.sizeDelta = new Vector2(0, 24);
_fuelFill = fillGo.GetComponent&lt;Image&gt;();
_fuelFill.sprite = _square;
_fuelFill.type = Image.Type.Filled;
_fuelFill.fillMethod = Image.FillMethod.Horizontal;
_fuelFill.color = FurnaceOrange;
//        ·
_firewoodButton = MakeActionButton("FirewoodButton", "  ", new Vector2(-260, -420), FirewoodRequested, out _
//
_buildButton = MakeActionButton("BuildButton", " \n  ", new Vector2(300, -420), BuildRequested, out _buildIm
//
_iceButton = MakeActionButton("IceButton", "  \n ", new Vector2(-300, -660), DigRequested, out _iceImage);
//        M4          /         ·  ×2·     ——
_nearButton = MakeActionButton("NearNode", "  ·  \n+6   ", new Vector2(170, -470), NearDispatchRequested, ou
_farButton = MakeActionButton("FarNode", "  ·  \n+12   15%", new Vector2(450, -260), FarDispatchRequested, o
// X027 generated art: furnace + building paintings on the action blocks.
ApplyActionArt(_furnaceRt.GetComponent&lt;Image&gt;(), "P05_icon_fire");
ApplyActionArt(_firewoodImage, "P05_icon_wood");
ApplyActionArt(_buildImage, "P05_building_01_wood");
ApplyActionArt(_iceImage, "P05_building_02_ice");
ApplyActionArt(_nearImage, "P05_building_03_farm");
ApplyActionArt(_farImage, "P05_building_04_iron");
//
var digBar = MakeCanvasChild("DigBar");
digBar.anchoredPosition = new Vector2(-300, -540);
digBar.sizeDelta = new Vector2(220, 18);
var digBg = digBar.GetComponent&lt;Image&gt;();
digBg.sprite = _square;
digBg.color = new Color(0.15f, 0.18f, 0.22f);
var digFillGo = new GameObject("Fill", typeof(RectTransform), typeof(Image));
digFillGo.transform.SetParent(digBar, false);
var digFillRt = (RectTransform)digFillGo.transform;
digFillRt.anchorMin = new Vector2(0f, 0.5f);
digFillRt.anchorMax = new Vector2(1f, 0.5f);
```

```
        digFillRt.anchoredPosition = Vector2.zero;
        digFillRt.sizeDelta = new Vector2(0, 18);
        _digFill = digFillGo.GetComponent&lt;Image&gt;();
        _digFill.sprite = _square;
        _digFill.type = Image.Type.Filled;
        _digFill.fillMethod = Image.FillMethod.Horizontal;
        _digFill.color = IceBlue;
        digBar.gameObject.SetActive(false);
        //               ——"    "
        _digStageText = MakeText(_canvas.transform, "", 34, new Vector2(-300, -450), TextAnchor.MiddleCenter, 340, 6
        _digStageText.color = new Color(0.05f, 0.09f, 0.15f);   // U184: deep-ink - was IceBlue-on-snow 1.4:1 (17 do
        _digStageText.gameObject.SetActive(false);
        //
        var dispatchBar = MakeCanvasChild("DispatchBar");
        dispatchBar.anchoredPosition = new Vector2(310, -100);
        dispatchBar.sizeDelta = new Vector2(220, 18);
        var dispBg = dispatchBar.GetComponent&lt;Image&gt;();
        dispBg.sprite = _square;
        dispBg.color = new Color(0.15f, 0.18f, 0.22f);
        var dispFillGo = new GameObject("Fill", typeof(RectTransform), typeof(Image));
        dispFillGo.transform.SetParent(dispatchBar, false);
        var dispFillRt = (RectTransform)dispFillGo.transform;
        dispFillRt.anchorMin = new Vector2(0f, 0.5f);
        dispFillRt.anchorMax = new Vector2(1f, 0.5f);
        dispFillRt.anchoredPosition = Vector2.zero;
        dispFillRt.sizeDelta = new Vector2(0, 18);
        _dispatchFill = dispFillGo.GetComponent&lt;Image&gt;();
        _dispatchFill.sprite = _square;
        _dispatchFill.type = Image.Type.Filled;
        _dispatchFill.fillMethod = Image.FillMethod.Horizontal;
        _dispatchFill.color = IceBlue;
        dispatchBar.gameObject.SetActive(false);
        //        M5
        _socialButton = MakeActionButton("SocialButton", "    \n  ", new Vector2(380, 300), SocialTapped, out _socia
        _socialButton.gameObject.SetActive(false);
    }
    Image _socialImage;
    void BuildHud()
    {
        var root = _canvas.transform;
        _coinText = MakeText(root, "   0", 46, new Vector2(0, 880), TextAnchor.UpperLeft, 360, 70);
        _coinText.color = SnowWhite;
        _fuelText = MakeText(root, "   20%", 46, new Vector2(0, 880), TextAnchor.UpperCenter, 380, 70);
        _fuelText.color = FurnaceOrange;
        _citizenText = MakeText(root, "   1", 44, new Vector2(0, 820), TextAnchor.UpperRight, 320, 60);
        _citizenText.color = SnowWhite;
        _woodText = MakeText(root, "   3", 44, new Vector2(0, 760), TextAnchor.UpperRight, 320, 60);
        _woodText.color = SnowWhite;   // U184: was IceGray-on-deep ~2.6:1 (17 doc section 3.7)
        _hintText = MakeText(root, "", 40, new Vector2(0, -840), TextAnchor.MiddleCenter, 1020, 150);
        _hintText.color = new Color(0.05f, 0.09f, 0.15f);   // U184: deep-ink on snow band - was SnowWhite-on-snow 1
```

```
}
void BuildOfflinePopup()
{
    _offlinePopup = MakeOverlayPanel("OfflinePopup", new Color(0.06f, 0.10f, 0.16f, 0.94f));
    MakeText(_offlinePopup, "   8   ", 66, new Vector2(0, 220), TextAnchor.MiddleCenter, 900, 100).color = SnowW
    MakeText(_offlinePopup, "   ×240    ", 52, new Vector2(0, 80), TextAnchor.MiddleCenter, 900, 90).color = Ice
    MakePanelButton(_offlinePopup, "  ", new Vector2(0, -160), () =&gt; ClaimRequested?.Invoke());
}
void BuildSettlePanel()
{
    _settlePanel = MakeOverlayPanel("SettlePanel", new Color(0.06f, 0.10f, 0.16f, 0.96f));
    MakeText(_settlePanel, "      ", 96, new Vector2(0, 560), TextAnchor.MiddleCenter, 1040, 120).color = Furnac
    _settleStats = MakeText(_settlePanel, "", 44, new Vector2(0, 360), TextAnchor.MiddleCenter, 1020, 380);
    MakePanelButton(_settlePanel, "    ", new Vector2(-260, -300), () =&gt; UnityEngine.SceneManagement.SceneMan
    MakePanelButton(_settlePanel, "     ", new Vector2(260, -300), () =&gt; UnityEngine.SceneManagement.SceneMan
    _settlePanel.gameObject.SetActive(false);
}
// ————      ————
public void SyncView(FurnaceSim sim)
{
    var phase = sim.Phase;
    if (phase != _lastPhase)
    {
        _lastPhase = phase;
        OnPhaseVisual(phase);
    }
    float fuel01 = sim.Fuel / 100f;
    _fuelFill.fillAmount = fuel01;
    //      →        ① +
    float target = NewbieConfig.LightRadiusByLevel[sim.FireLevel] + fuel01 * 0.05f;
    _circleScale = Mathf.Lerp(_circleScale, target, 0.08f);
    _lightCircle.localScale = Vector3.one * _circleScale;
    if (phase == FurnaceSim.SimPhase.Digging)
        _digFill.fillAmount = sim.PhaseTimer / NewbieConfig.DigSeconds;
    if (phase == FurnaceSim.SimPhase.Dispatching)
        _dispatchFill.fillAmount = sim.PhaseTimer / NewbieConfig.DispatchSeconds;
    SetButtonState(_firewoodButton, _firewoodImage, phase == FurnaceSim.SimPhase.AwaitFirewood);
    SetButtonState(_buildButton, _buildImage, phase == FurnaceSim.SimPhase.AwaitBuild);
    SetButtonState(_iceButton, _iceImage, phase == FurnaceSim.SimPhase.AwaitDig);
    bool dispatchable = phase == FurnaceSim.SimPhase.AwaitDispatch;
    SetButtonState(_nearButton, _nearImage, dispatchable);
    SetButtonState(_farButton, _farImage, dispatchable);
}
void OnPhaseVisual(FurnaceSim.SimPhase phase)
{
    switch (phase)
    {
        case FurnaceSim.SimPhase.AwaitClaim:
            _offlinePopup.gameObject.SetActive(true);
            break;
```

```
            case FurnaceSim.SimPhase.AwaitFirewood:
                _offlinePopup.gameObject.SetActive(false);
                break;
            case FurnaceSim.SimPhase.Digging:
                _digFill.transform.parent.gameObject.SetActive(true);
                _digStageText.gameObject.SetActive(true);
                break;
            case FurnaceSim.SimPhase.AwaitDispatch:
                _digFill.transform.parent.gameObject.SetActive(false);
                _digStageText.gameObject.SetActive(false);
                break;
            case FurnaceSim.SimPhase.Dispatching:
                _dispatchFill.transform.parent.gameObject.SetActive(true);
                break;
            case FurnaceSim.SimPhase.Reveal:
                _dispatchFill.transform.parent.gameObject.SetActive(false);
                _socialButton.gameObject.SetActive(true);
                break;
            case FurnaceSim.SimPhase.Settled:
                _socialButton.gameObject.SetActive(true);
                break;
        }
    }
    static void SetButtonState(Button button, Image image, bool active)
    {
        if (button.interactable == active) return;
        button.interactable = active;
        image.color = active ? FurnaceOrange : new Color(0.30f, 0.34f, 0.40f);
    }
    // ——— HUD    ———
    public void SetCoins(int c) =&gt; _coinText.text = "   " + c;
    public void SetCitizens(int n) =&gt; _citizenText.text = "   " + n;
    public void SetWood(int w) =&gt; _woodText.text = "   " + w;
    public void SetFuel(int f) =&gt; _fuelText.text = "   " + f + "%";
    public void SetHint(string hint) =&gt; _hintText.text = hint;
    /// &lt;summary&gt;             →   →   →    &lt;/summary&gt;
    public void SetDigStage(int stage)
    {
        _digStageText.text = NewbieConfig.DigStages[stage - 1] + " " + stage * 25 + "% ";
        // X027: frozen silhouette peek at "reveal", full-color awake at "awake".
        if (_digPortrait == null) return;
        var art = stage &gt;= 4 ? Sy.Core.ArtRuntime.Get("P05_general_02_guanyu_awake")
            : stage == 3 ? Sy.Core.ArtRuntime.Get("P05_general_02_guanyu_frozen")
            : null;
        if (art != null)
        {
            _digPortrait.sprite = art;
            _digPortrait.color = stage &gt;= 4 ? Color.white : new Color(1f, 1f, 1f, 0.6f);
            _digPortrait.preserveAspect = true;
            _digPortrait.gameObject.SetActive(true);
```

```
    }
    else
    {
        _digPortrait.gameObject.SetActive(false);
    }
}
public void HideIntro()
{
    // M0     Hint
}
public void ShowSettle(int coins, int citizens, int wood, int fuel, int day, bool outside)
{
    _settlePanel.gameObject.SetActive(true);
    _settleStats.text = $"   {coins} ·    {citizens} ·    {wood} ·    {fuel}%\n    {day}     {NewbieConfig.Miles
}
// ─────      ─────
/// &lt;summary&gt;X027: swap a solid action block for generated art when the key exists.&lt;/summary&gt;
static void ApplyActionArt(Image image, string key)
{
    var art = Sy.Core.ArtRuntime.Get(key);
    if (image == null || art == null) return;
    image.sprite = art;
    image.color = Color.white;
    image.preserveAspect = true;
}
IEnumerator SnowRoutine()
{
    var flakes = new RectTransform[7];
    for (int i = 0; i &lt; flakes.Length; i++)
    {
        var go = new GameObject("Snow_" + i, typeof(RectTransform), typeof(Image));
        go.transform.SetParent(_canvas.transform, false);
        flakes[i] = (RectTransform)go.transform;
        flakes[i].anchorMin = flakes[i].anchorMax = new Vector2(0.5f, 0.5f);
        flakes[i].sizeDelta = new Vector2(18, 18);
        flakes[i].anchoredPosition = new Vector2(UnityEngine.Random.Range(-520f, 520f), UnityEngine.Random.Range
        var img = go.GetComponent&lt;Image&gt;();
        img.sprite = _square;
        img.color = new Color(SnowWhite.r, SnowWhite.g, SnowWhite.b, 0.75f);
        flakes[i].SetAsFirstSibling();
        flakes[i].SetSiblingIndex(1);
    }
    while (true)
    {
        for (int i = 0; i &lt; flakes.Length; i++)
        {
            var pos = flakes[i].anchoredPosition;
            pos.y -= 60f * Time.deltaTime;
            pos.x += Mathf.Sin(Time.time + i) * 10f * Time.deltaTime;
            if (pos.y &lt; -950f)
```

```
            {
                pos.y = 950f;
                pos.x = UnityEngine.Random.Range(-520f, 520f);
            }
            flakes[i].anchoredPosition = pos;
        }
        yield return null;
    }
}
IEnumerator FlameRoutine()
{
    while (true)
    {
        float fuel01 = _sim.Fuel / 100f;
        float pulse = 0.5f + 0.5f * Mathf.Sin(Time.unscaledTime * (2f + fuel01 * 3f));
        _furnaceRt.localScale = Vector3.one * (1f + 0.05f + 0.06f * pulse * fuel01);
        yield return null;
    }
}
// ─────    ─────
RectTransform MakeCanvasChild(string name)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Image));
    go.transform.SetParent(_canvas.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    return rt;
}
RectTransform MakeFullImage(string name, Color color)
{
    var rt = MakeCanvasChild(name);
    rt.anchorMin = Vector2.zero;
    rt.anchorMax = Vector2.one;
    rt.anchoredPosition = Vector2.zero;
    rt.sizeDelta = Vector2.zero;
    rt.GetComponent&lt;Image&gt;().color = color;
    return rt;
}
Button MakeActionButton(string name, string label, Vector2 pos, Action onClick, out Image image)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Image), typeof(Button));
    go.transform.SetParent(_canvas.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = pos;
    rt.sizeDelta = new Vector2(160, 160);
    image = go.GetComponent&lt;Image&gt;();
    image.sprite = _square;
    image.color = new Color(0.30f, 0.34f, 0.40f);
    var label1 = MakeText(go.transform, label, 34, Vector2.zero, TextAnchor.MiddleCenter, 170, 170);
```

```
        label1.color = SnowWhite;
        var button = go.GetComponent&lt;Button&gt;();
        button.interactable = false;
        button.onClick.AddListener(() =&gt; onClick());
        return button;
    }
    Button MakePanelButton(RectTransform parent, string label, Vector2 pos, Action onClick)
    {
        var go = new GameObject(label, typeof(RectTransform), typeof(Image), typeof(Button));
        go.transform.SetParent(parent, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = pos;
        rt.sizeDelta = new Vector2(420, 120);
        var image = go.GetComponent&lt;Image&gt;();
        image.sprite = _square;
        image.color = FurnaceOrange;
        MakeText(go.transform, label, 48, Vector2.zero, TextAnchor.MiddleCenter, 420, 120).color = InkBlue;
        var button = go.GetComponent&lt;Button&gt;();
        button.onClick.AddListener(() =&gt; onClick());
        return button;
    }
    RectTransform MakeOverlayPanel(string name, Color color)
    {
        var go = new GameObject(name, typeof(RectTransform), typeof(Image));
        go.transform.SetParent(_canvas.transform, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = Vector2.zero;
        rt.anchorMax = Vector2.one;
        rt.offsetMin = rt.offsetMax = Vector2.zero;
        var image = go.GetComponent&lt;Image&gt;();
        image.sprite = _square;
        image.color = color;
        return rt;
    }
    static Text MakeText(Transform parent, string text, int size, Vector2 pos, TextAnchor anchor, float w, float h)
    {
        var go = new GameObject("Text", typeof(RectTransform), typeof(Text));
        go.transform.SetParent(parent, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = pos;
        rt.sizeDelta = new Vector2(w, h);
        var t = go.GetComponent&lt;Text&gt;();
        t.font = Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf");
        t.text = text;
        t.fontSize = size;
        t.alignment = anchor;
        t.color = new Color(0.91f, 0.89f, 0.85f);
        t.supportRichText = true;
```

```
            return t;
        }
        static Sprite MakeSquareSprite()
        {
            var tex = new Texture2D(8, 8, TextureFormat.RGBA32, false);
            var px = tex.GetPixels32();
            for (int i = 0; i &lt; px.Length; i++) px[i] = new Color32(255, 255, 255, 255);
            tex.SetPixels32(px);
            tex.Apply(false, false);
            return Sprite.Create(tex, new Rect(0, 0, 8, 8), new Vector2(0.5f, 0.5f), 8f);
        }
    }
}
// NewbieConfig.cs
namespace Sy.P05.Newbie
{
    /// &lt;summary&gt;
    /// P05         ·      V0.2   ——            X009+IConfigService      Design/configs/P05 JSON
    ///       V1.3§4.1  ticking  (fuelPerMin="4+fireLv*2"·       )/§4.2   ①  =    (  →  /
    ///        ×2·frostRisk="0.05*distFromFire")/②       (   →   →   →  ——"4h  " "    ")/
    ///    D1-D2  /    +    06    05 M0-M5
    ///      #F7F9FC ×   #A8C5DA ×    #1E3A5F ×    #FF6B1A ×   #6B7B8C
    ///         10  →10      →10s            §4.2
    /// &lt;/summary&gt;
    public static class NewbieConfig
    {
        public const string GameTitle = "   ";
        public const string GameSlogan = "          ";
        public const float IntroSeconds = 3f;            // M0   +
        public const float DigSeconds = 10f;            // M3         10  ·
        public const float DispatchSeconds = 10f;      // M4
        public const float RevealSeconds = 2f;         // M5
        public const float MaxTutorialSeconds = 210f;  //      ≤3.5min 06  ②
        // ——          §4.1 fuelPerMin="4+fireLv*2"  ——
        public const float StartFuel = 20f;              //      M0
        public const float FuelFloor = 1f;               //
        public const float FirewoodFuel = 30f;           // M1     +30   1
        public const int StartWood = 3;                  //
        public const float WoodyardWoodInterval = 3f;    //   lv1    3s+1
        public const int WoodyardWoodCap = 5;           //
        /// &lt;summary&gt;      fuelPerMin="4+fireLv*2"   =  /60  &lt;/summary&gt;
        public static float FuelDrainPerSecond(int fireLevel) =&gt; (4f + fireLevel * 2f) / 60f;
        // ——   =        §4.2   ①     →        =     ——
        public static readonly float[] LightRadiusByLevel = { 0f, 0.55f, 0.95f };   // [lv1][lv2]
        /// &lt;summary&gt;      0=          1=        ·  ×2  &lt;/summary&gt;
        public static readonly string[] DispatchNames = { "  ·  ", "  ·  " };
        public static readonly int[] DispatchYield = { 6, 12 };                     //     ×2
        public static readonly float[] DispatchFrostRisk = { 0f, 0.15f };           // frostRisk=0.05*        15%
        public const float OutsideWarmupFuel = 5f;       //               ——
        // ——          V1.2  ②       +"    "  ——
```

```
        public static readonly string[] DigStages =
        {
            "   ——      ",
            "   ——      ",
            "   ——      ",
            "  ——     ",
        };
        // ——        §4.1     +      D1-D2    ——
        public const int StartDay = 1;
        public const string ColdWaveTeaser = "      ——D1-D2           D3   ";
        public const int MilestoneDay = 7;                //  7
        public const int OfflineCoins = 240;             // M1    8h
        public const int StartCitizens = 1;             //           →2
    }
}
// TutorialFlow.cs
using System;
namespace Sy.P05.Newbie
{
    /// &lt;summary&gt;
    ///        evt_newbie_step 1-5 ①  +  ②   ③     ④  +  ⑤     +
    ///      FurnaceSim                          /  /    ——
    /// Play   headless   NewbieFlowCheck
    /// &lt;/summary&gt;
    public sealed class TutorialFlow
    {
        public int Step { get; private set; }
        readonly Action&lt;string&gt; _hint;
        readonly Action&lt;int&gt; _onStep;
        public TutorialFlow(FurnaceSim sim, Action&lt;string&gt; hint, Action&lt;int&gt; onStep)
        {
            _hint = hint ?? delegate { };
            _onStep = onStep ?? delegate { };
            sim.PhaseChanged += OnPhaseChanged;
            sim.Claimed += () =&gt; _hint("            8h  ·        ——             1 ·  +30 ");
            sim.FirewoodAdded += OnFirewoodAdded;
            sim.WoodyardBuilt += OnWoodyardBuilt;
            sim.FireLevelUp += OnFireLevelUp;
            sim.DigStarted += () =&gt; _hint("   ……             10  ·      ");
            sim.DigStageReached += OnDigStageReached;
            sim.SurvivorDug += OnSurvivorDug;
            sim.DispatchStarted += OnDispatchStarted;
            sim.FrostRiskWarned += () =&gt; _hint("   ·       15% frostRisk      ——      5      ×2");
            sim.OrderDone += OnOrderDone;
            sim.Settled += OnSettled;
            _hint("        ——         ");
        }
        void Advance(int step)
        {
            if (Step &gt;= step) return;
```

```
        Step = step;
        _onStep(step);
    }
    void OnPhaseChanged(FurnaceSim.SimPhase phase)
    {
        switch (phase)
        {
            case FurnaceSim.SimPhase.AwaitClaim:
                _hint("  8  ——           ");
                break;
            case FurnaceSim.SimPhase.AwaitFirewood:
                _hint("          ——    +30");
                break;
            case FurnaceSim.SimPhase.AwaitBuild:
                _hint("        ——      lv1     ·    ·   lv2 ");
                break;
            case FurnaceSim.SimPhase.AwaitDig:
                _hint("        ——        ");
                break;
            case FurnaceSim.SimPhase.AwaitDispatch:
                _hint("         +1                      ×2");
                break;
            case FurnaceSim.SimPhase.Reveal:
                _hint("    ——             " + NewbieConfig.ColdWaveTeaser + " ");
                break;
        }
    }
    void OnFirewoodAdded()
    {
        Advance(1);
        _hint("          ——  +         ");
    }
    void OnWoodyardBuilt()
    {
        Advance(2);
        _hint("  lv1         3s+1 ·    5 ——      ");
    }
    void OnFireLevelUp()
    {
        _hint("   lv2     ——             ①     →     ");
    }
    void OnDigStageReached(int stage)
    {
        _hint(NewbieConfig.DigStages[stage - 1] + " " + stage * 25 + "% ");
    }
    void OnSurvivorDug()
    {
        Advance(3);
        _hint("         \"        \"——                    ·4-24h   ");
    }
```

```
        void OnDispatchStarted(int node)
        {
            Advance(4);
            _hint("        ——    " + (node == 1 ? "    " : "    ") + "  ");
        }
        void OnOrderDone(int yield, int node)
        {
            _hint("        +" + yield + (node == 1 ? "     ×2·       5 " : "      ") + "——    ");
        }
        void OnSettled()
        {
            Advance(5);
            _hint("        " + NewbieConfig.StartDay + " → " + (NewbieConfig.StartDay + 1)
                  + " —— " + NewbieConfig.MilestoneDay + "                 ");
        }
    }
}
// CampBuilder.cs
using System;
using UnityEngine;
namespace Sy.P05.Night
{
    /// &lt;summary&gt;
    ///      §   ·  C#    /     8*lv^1.5+  5*lv^1.2·§
    ///        0-3                      ResourceBank
    ///    /    /     /
    ///     Night   ——        FurnaceSim      =
    /// &lt;/summary&gt;
    public sealed class CampBuilder
    {
        const string LevelsKey = "p05_camp_levels";
        const string WorkersKey = "p05_camp_workers";
        const string DayKey = "p05_camp_day";
        public int[] Levels { get; } = new int[NightConfig.Buildings.Length];
        public int[] Workers { get; } = new int[NightConfig.Buildings.Length];
        public int Day { get; private set; } = 1;
        public static CampBuilder Load()
        {
            var c = new CampBuilder();
            string lv = PlayerPrefs.GetString(LevelsKey, "");
            if (lv.Length &gt; 0)
            {
                var parts = lv.Split(',');
                for (int i = 0; i &lt; parts.Length &amp;&amp; i &lt; c.Levels.Length; i++)
                    if (int.TryParse(parts[i], out int v)) c.Levels[i] = v;
            }
            string wk = PlayerPrefs.GetString(WorkersKey, "");
            if (wk.Length &gt; 0)
            {
                var parts = wk.Split(',');
```

```
            for (int i = 0; i &lt; parts.Length &amp;amp;&amp;amp; i &lt; c.Workers.Length; i++)
                if (int.TryParse(parts[i], out int v)) c.Workers[i] = v;
        }
        c.Day = PlayerPrefs.GetInt(DayKey, 1);
        return c;
    }
    void Save()
    {
        var lv = new System.Text.StringBuilder();
        var wk = new System.Text.StringBuilder();
        for (int i = 0; i &lt; Levels.Length; i++)
        {
            if (i &gt; 0) { lv.Append(','); wk.Append(','); }
            lv.Append(Levels[i]);
            wk.Append(Workers[i]);
        }
        PlayerPrefs.SetString(LevelsKey, lv.ToString());
        PlayerPrefs.SetString(WorkersKey, wk.ToString());
        PlayerPrefs.SetInt(DayKey, Day);
        PlayerPrefs.Save();
    }
    public void AdvanceDay() { Day++; Save(); }
    public int Level(int buildingId) =&gt; Levels[buildingId - 1];
    public bool IsBuilt(int buildingId) =&gt; Level(buildingId) &gt; 0;
    /// &lt;summary&gt;     =       +1   ·§   &lt;/summary&gt;
    public int WorkerSlots =&gt; Level(2);
    /// &lt;summary&gt;     0-3·         false=     &lt;/summary&gt;
    public bool AssignWorker(int buildingId, int count)
    {
        if (buildingId &lt; 1 || buildingId &gt; Levels.Length) return false;
        if (count &lt; 0 || count &gt; 3) return false;
        int used = 0;
        for (int i = 0; i &lt; Workers.Length; i++)
            if (i != buildingId - 1) used += Workers[i];
        if (used + count &gt; WorkerSlots) return false;
        Workers[buildingId - 1] = count;
        Save();
        return true;
    }
    /// &lt;summary&gt;      §    ·lv0→  1  =1               false &lt;/summary&gt;
    public bool TryBuildOrUpgrade(int buildingId, ResourceBank bank)
    {
        var def = Def(buildingId);
        if (def == null) return false;
        if (def.UnlockDay &gt; Day) return false;
        int current = Level(buildingId);
        if (current &gt;= 30) return false;
        int next = current + 1;
        int iron = NightConfig.BuildCostIron(next);
        int grain = NightConfig.BuildCostGrain(next);
```

```
            if (!bank.TrySpend(0f, grain, iron)) return false;
            Levels[buildingId - 1] = next;
            Save();
            return true;
        }
        public static NightConfig.BuildingDef Def(int buildingId)
        {
            foreach (var b in NightConfig.Buildings)
                if (b.Id == buildingId) return b;
            return null;
        }
        // ————      §     ·   ·  /         ————
        public (float wood, float grain, float iron) ProductionPerMinute(float heroGatherMul)
        {
            float mul = heroGatherMul * HeroShrineBonus;
            float wood = NightConfig.WoodPerMin(Level(1), Workers[0]) * mul;
            float grain = NightConfig.GrainPerMin(Level(3), Workers[2]) * mul;
            float iron = NightConfig.IronPerMin(Level(4), Workers[3]) * mul;
            return (wood, grain, iron);
        }
        public int WarehouseLevel =&gt; Level(6);
        public int DigQueueSize
        {
            get
            {
                int size = Level(7) &gt; 0 ? 1 + Level(7) / 5 : 0;
                return size;
            }
        }
        /// &lt;summary&gt;          -5%/ · 5 =25%·§   &lt;/summary&gt;
        public float WatchTowerReduction
            =&gt; System.Math.Min(NightConfig.WatchTowerMaxLevel,
                Level(9)) * NightConfig.WatchTowerReductionPerLevel;
        /// &lt;summary&gt;        +10%/5 ·§   &lt;/summary&gt;
        public float HeroShrineBonus =&gt; Level(10) &gt; 0 ? 1f + Level(10) / 5 * 0.10f : 1f;
        public static void ClearForTests()
        {
            PlayerPrefs.DeleteKey(LevelsKey);
            PlayerPrefs.DeleteKey(WorkersKey);
            PlayerPrefs.DeleteKey(DayKey);
        }
    }
}
// ColdWave.cs
using System;
using UnityEngine;
namespace Sy.P05.Night
{
    /// &lt;summary&gt;
    ///      § ·  C#          →   →   +
```

```
///     -15%/ ·   2    ·  3          +
///  ≥48h   →     +   =   30%   +   /
///      =  ×2×30    §  burnMultiplier×nightMinutes ——
///      =  →         Night   ·        FurnaceSim
/// &lt;/summary&gt;
public sealed class ColdWave
{
    public enum Phase { Idle, Warning, Night, Settled }
    const string LastTickKey = "p05_coldwave_last_ms";
    const string PenaltyNightsKey = "p05_coldwave_penalty_nights";
    /// &lt;summary&gt;         ·       1  ·   2h——§  subscribeAlarm     2h    &lt;/summary&gt;
    public const float CycleHours = 22f;
    public const float WarningHours = 2f;
    public const float NightHours = NightConfig.NightMinutes / 60f;
    public Phase CurrentPhase(long nowMs, long lastCwMs)
    {
        long elapsed = nowMs - lastCwMs;
        if (elapsed &lt; 0) return Phase.Idle;
        float hours = elapsed / 3_600_000f;
        float cyclePos = hours % CycleHours;
        if (cyclePos &lt; CycleHours - NightHours - WarningHours) return Phase.Idle;
        if (cyclePos &lt; CycleHours - NightHours) return Phase.Warning;
        if (cyclePos &lt; CycleHours) return Phase.Night;
        return Phase.Idle;
    }
    public int PenaltyNights { get; private set; }
    public static ColdWave Load()
    {
        return new ColdWave
        {
            PenaltyNights = PlayerPrefs.GetInt(PenaltyNightsKey, 0),
        };
    }
    void Save()
    {
        PlayerPrefs.SetInt(PenaltyNightsKey, PenaltyNights);
        PlayerPrefs.Save();
    }
    /// &lt;summary&gt;       §    =    ×2×30min·   /      &lt;/summary&gt;
    public static float NightWoodNeed(int fireLevel, CampBuilder camp, HeroDig heroes)
    {
        float burn = NightConfig.FuelPerMin(Mathf.Max(1, fireLevel)) * NightConfig.NightBurnMultiplier;
        float minutes = NightConfig.NightMinutes;
        float reduction = camp.WatchTowerReduction + heroes.ColdConsumptionReduction();
        reduction = Mathf.Min(0.60f, reduction);   //     60%
        return burn * minutes * (1f - reduction);
    }
    /// &lt;summary&gt;        =     =  →  +1       3      -15%       &lt;/summary&gt;
    public bool SurviveNight(ResourceBank bank, float need)
    {
```

```
            if (bank.Wood &gt;= need)
            {
                bank.TrySpend(need, 0f, 0f);
                return true;
            }
            bank.TrySpend(bank.Wood, 0f, 0f);   //
            if (PenaltyNights &lt; NightConfig.FailPenaltyCapNights)
            {
                PenaltyNights++;
                Save();
            }
            return false;
        }
        /// &lt;summary&gt;     §  ≥48h      +       30%  ·      &lt;/summary&gt;
        public bool TryExempt(long absentMs, ResourceBank bank, int fireLevel, CampBuilder camp)
        {
            if (absentMs &lt; NightConfig.ExemptAbsentHours * 3_600_000L) return false;
            float gift = NightConfig.FurnaceCapacity(Mathf.Max(1, fireLevel)) * NightConfig.ExemptReturnWoodPct;
            bank.AddWood(gift);
            return true;
        }
        /// &lt;summary&gt;       0.85^     ·   2  →   0.7225   3      -15%     &lt;/summary&gt;
        public float ProductionPenalty()
        {
            int nights = Mathf.Min(PenaltyNights, NightConfig.FailPenaltyCapNights);
            if (nights &lt;= 0) return 1f;
            return 1f - nights * NightConfig.FailPenaltyRate;
        }
        public void ClearPenalty()
        {
            PenaltyNights = 0;
            Save();
        }
        public static void Stamp(long nowMs)
        {
            PlayerPrefs.SetString(LastTickKey, nowMs.ToString());
            PlayerPrefs.Save();
        }
        public static long LastStamp()
        {
            var raw = PlayerPrefs.GetString(LastTickKey, "");
            return long.TryParse(raw, out long v) ? v : 0;
        }
        public static void ClearForTests()
        {
            PlayerPrefs.DeleteKey(LastTickKey);
            PlayerPrefs.DeleteKey(PenaltyNightsKey);
        }
    }
}
```

```
// HeroDig.cs
using System;
using UnityEngine;
namespace Sy.P05.Night
{
    /// &lt;summary&gt;
    ///      § ·  C#                    4h-24h
    ///      P0-1       &amp;lt; min(60min, 20%×   )       frozen→awake
    ///      =    1+ 5 +   +1    12 PlayerPrefs
    /// &lt;/summary&gt;
    public sealed class HeroDig
    {
        public enum HeroState { Locked, Digging, Awake }
        const string StateKey = "p05_hero_states";
        const string ProgressKey = "p05_hero_progress";
        public HeroState[] States { get; } = new HeroState[NightConfig.Heroes.Length];
        /// &lt;summary&gt;        ——  /       &lt;/summary&gt;
        public float[] Progress { get; } = new float[NightConfig.Heroes.Length];
        public static HeroDig Load()
        {
            var d = new HeroDig();
            string st = PlayerPrefs.GetString(StateKey, "");
            if (st.Length &gt; 0)
            {
                var parts = st.Split(',');
                for (int i = 0; i &lt; parts.Length &amp;&amp; i &lt; d.States.Length; i++)
                    if (int.TryParse(parts[i], out int v)) d.States[i] = (HeroState)v;
            }
            string pg = PlayerPrefs.GetString(ProgressKey, "");
            if (pg.Length &gt; 0)
            {
                var parts = pg.Split(',');
                for (int i = 0; i &lt; parts.Length &amp;&amp; i &lt; d.Progress.Length; i++)
                    if (float.TryParse(parts[i], System.Globalization.NumberStyles.Float,
                        System.Globalization.CultureInfo.InvariantCulture, out float v)) d.Progress[i] = v;
            }
            return d;
        }
        void Save()
        {
            var st = new System.Text.StringBuilder();
            var pg = new System.Text.StringBuilder();
            for (int i = 0; i &lt; States.Length; i++)
            {
                if (i &gt; 0) { st.Append(','); pg.Append(','); }
                st.Append((int)States[i]);
                pg.Append(Progress[i].ToString("0.###",
                    System.Globalization.CultureInfo.InvariantCulture));
            }
            PlayerPrefs.SetString(StateKey, st.ToString());
```

```
        PlayerPrefs.SetString(ProgressKey, pg.ToString());
        PlayerPrefs.Save();
    }
    public int DiggingCount()
    {
        int n = 0;
        for (int i = 0; i &lt; States.Length; i++)
            if (States[i] == HeroState.Digging) n++;
        return n;
    }
    public static NightConfig.HeroDef Def(int heroId)
    {
        foreach (var h in NightConfig.Heroes)
            if (h.Id == heroId) return h;
        return null;
    }
    /// &lt;summary&gt;        ResourceBank  ·     CampBuilder     &lt;/summary&gt;
    public bool StartDig(int heroId, ResourceBank bank, CampBuilder camp)
    {
        int at = heroId - 1;
        if (at &lt; 0 || at &gt;= States.Length) return false;
        if (States[at] != HeroState.Locked) return false;
        var def = Def(heroId);
        if (def == null) return false;
        if (camp.DigQueueSize &lt;= DiggingCount()) return false;
        if (!bank.TrySpend(0f, 0f, def.PickCost)) return false;
        States[at] = HeroState.Digging;
        Progress[at] = 0f;
        Save();
        return true;
    }
    /// &lt;summary&gt;   dtHours·        tick       →   &lt;/summary&gt;
    public void Tick(float dtHours)
    {
        if (dtHours &lt;= 0f) return;
        bool changed = false;
        for (int i = 0; i &lt; States.Length; i++)
        {
            if (States[i] != HeroState.Digging) continue;
            var def = NightConfig.Heroes[i];
            Progress[i] += dtHours;
            if (Progress[i] &gt;= def.DigHours)
            {
                States[i] = HeroState.Awake;
                Progress[i] = def.DigHours;
            }
            changed = true;
        }
        if (changed) Save();
    }
```

```
/// &lt;summary&gt;P0-1           &amp;lt; min(1h, 20%×   ) &lt;/summary&gt;
public bool BoostWindowOpen(int heroId)
{
    int at = heroId - 1;
    if (at &lt; 0 || at &gt;= States.Length || States[at] != HeroState.Digging) return false;
    var def = NightConfig.Heroes[at];
    float remaining = def.DigHours - Progress[at];
    return remaining &lt;= NightConfig.DigBoostWindowHours(def.DigHours);
}
/// &lt;summary&gt;                    id -1=   &lt;/summary&gt;
public int TryBoostComplete(int heroId)
{
    if (!BoostWindowOpen(heroId)) return -1;
    int at = heroId - 1;
    States[at] = HeroState.Awake;
    Progress[at] = NightConfig.Heroes[at].DigHours;
    Save();
    return heroId;
}
public int AwakeCount()
{
    int n = 0;
    for (int i = 0; i &lt; States.Length; i++)
        if (States[i] == HeroState.Awake) n++;
    return n;
}
/// &lt;summary&gt;              +20%·  +15%           &lt;/summary&gt;
public float GatherMultiplier()
{
    float mul = 1f;
    for (int i = 0; i &lt; States.Length; i++)
    {
        if (States[i] != HeroState.Awake) continue;
        var def = NightConfig.Heroes[i];
        if (def.Id == 1) mul += 0.20f;      //      +20%
        if (def.Id == 4) mul += 0.15f;      //      +15%
    }
    return mul;
}
/// &lt;summary&gt;            -10%  &lt;/summary&gt;
public float ColdConsumptionReduction()
{
    for (int i = 0; i &lt; States.Length; i++)
    {
        if (States[i] != HeroState.Awake) continue;
        if (NightConfig.Heroes[i].Id == 2) return 0.10f;
    }
    return 0f;
}
/// &lt;summary&gt;       +10% &lt;/summary&gt;
```

```
        public float OfflineBonus()
        {
            for (int i = 0; i &lt; States.Length; i++)
            {
                if (States[i] != HeroState.Awake) continue;
                if (NightConfig.Heroes[i].Id == 6) return 0.10f;
            }
            return 0f;
        }
        public static void ClearForTests()
        {
            PlayerPrefs.DeleteKey(StateKey);
            PlayerPrefs.DeleteKey(ProgressKey);
        }
    }
}
// NightConfig.cs
namespace Sy.P05.Night
{
    /// &lt;summary&gt;
    ///     V1.4        X032       ·CEO   O-20261010-1245
    ///   =05_   .md v0.9 2026-09-16 +      V1.3 schema——
    ///                P0-1     =remaining&amp;lt;min(60min,20%×   )
    /// P1-1      lv1-2
    ///      =   1:1 1       =         8h     30
    /// &lt;/summary&gt;
    public static class NightConfig
    {
        // ———— §     fireLv 1-30     ·   7        ————
        public const int MaxFireLevel = 30;
        public static float FuelPerMin(int fireLv) =&gt; 4f + fireLv * 2f;          // §    /
        public static float FurnaceCapacity(int fireLv) =&gt; 240f * fireLv;       // §
        public static int LightRadius(int fireLv) =&gt; 1 + fireLv / 3;           // §
        // ———— §        ————
        public const int NightMinutes = 30;                 // §  nightMinutes
        public const float NightBurnMultiplier = 2.0f;        // §  burnMultiplier
        public const float FailPenaltyRate = 0.15f;           // §  production -15%/
        public const int FailPenaltyCapNights = 2;             // §     2    3     -15%
        public const float ExemptAbsentHours = 48f;           // §     ≥48h
        public const float ExemptReturnWoodPct = 0.30f;       // §     =   30%
        public const float SubscribeAlarmPct = 0.25f;         // §    &lt;25%
        public const int WatchTowerMaxLevel = 5;              // §
        public const float WatchTowerReductionPerLevel = 0.05f; // §  -5%/    -25%
        // ———— §          .    .         ————
        public static int BuildCostIron(int lv) =&gt; (int)System.Math.Round(8f * Pow15(lv));
        public static int BuildCostGrain(int lv) =&gt; (int)System.Math.Round(5f * Pow12(lv));
        static float Pow15(int lv) { float v = lv; return v * Powf(lv, 0.5f); }   // lv^1.5
        static float Pow12(int lv) { float v = lv; return v * Powf(lv, 0.2f); }   // lv^1.2
        static float Powf(float v, float e) =&gt; (float)System.Math.Pow(v, e);
        // ———— §      V1.4       =  /  /  /    4    ·   LiveOps     ————
```

```
public sealed class BuildingDef
{
    public int Id;
    public string Name;
    public int UnlockDay;
    public bool FirstWave;     // V1.4
    public string Effect;      //
}
public static readonly BuildingDef[] Buildings =
{
    new BuildingDef { Id = 1, Name = "  ",   UnlockDay = 0,  FirstWave = true,  Effect = "  3× × / " },
    new BuildingDef { Id = 2, Name = "  ",   UnlockDay = 0,  FirstWave = false, Effect = "  +1    " },
    new BuildingDef { Id = 3, Name = "  ",   UnlockDay = 1,  FirstWave = true,  Effect = "  2× × / " },
    new BuildingDef { Id = 4, Name = "  ",   UnlockDay = 2,  FirstWave = true,  Effect = "  1.5× × / " },
    new BuildingDef { Id = 5, Name = "  ",   UnlockDay = 3,  FirstWave = false, Effect = "  +2      " },
    new BuildingDef { Id = 6, Name = "  ",   UnlockDay = 5,  FirstWave = false, Effect = "     +500× " },
    new BuildingDef { Id = 7, Name = "   ", UnlockDay = 7,  FirstWave = true,  Effect = " 5     +1" },
    new BuildingDef { Id = 8, Name = "    ", UnlockDay = 10, FirstWave = false, Effect = " 5     +10%" },
    new BuildingDef { Id = 9, Name = "   ", UnlockDay = 15, FirstWave = false, Effect = "    -5%/ " },
    new BuildingDef { Id = 10, Name = "   ", UnlockDay = 20, FirstWave = false, Effect = " 5     +10%" },
};
// §       1  =1.0 / 2  =1.5 / 3  =2.0
public static float WorkerBonus(int workers) =&gt; workers switch
{
    0 =&gt; 0f,
    1 =&gt; 1f,
    2 =&gt; 1.5f,
    _ =&gt; 2f,
};
// §             ·lv×
public static float WoodPerMin(int woodyardLv, int workers) =&gt; 3f * woodyardLv * WorkerBonus(workers);
public static float GrainPerMin(int farmLv, int workers) =&gt; 2f * farmLv * WorkerBonus(workers);
public static float IronPerMin(int mineLv, int workers) =&gt; 1.5f * mineLv * WorkerBonus(workers);
public const float StorageBase = 500f;           // §
public const float WarehouseCapPerLevel = 500f;  // §       +500
public const int SurvivorRationsPerDay = 50;     // §       =50  / /
// ——— §             ·    ·V1.4       /      ———
public sealed class HeroDef
{
    public int Id;
    public string Name;
    public float DigHours;
    public int PickCost;
    public bool FirstWave;   // V1.4
    public string Function;  //
}
public static readonly HeroDef[] Heroes =
{
    new HeroDef { Id = 1, Name = "  ",   DigHours = 4f,  PickCost = 20, FirstWave = true,  Function = "  +20%
    new HeroDef { Id = 2, Name = "  ",   DigHours = 8f,  PickCost = 30, FirstWave = true,  Function = "       -1
```

```
            new HeroDef { Id = 3, Name = "   ", DigHours = 12f, PickCost = 35, FirstWave = false, Function = "    +1" },
            new HeroDef { Id = 4, Name = "  ",   DigHours = 16f, PickCost = 40, FirstWave = false, Function = "  +15%·
            new HeroDef { Id = 5, Name = "  ",   DigHours = 18f, PickCost = 45, FirstWave = false, Function = "    +15%"
            new HeroDef { Id = 6, Name = "  ",   DigHours = 20f, PickCost = 50, FirstWave = false, Function = "    +10%"
            new HeroDef { Id = 7, Name = "  ",   DigHours = 22f, PickCost = 55, FirstWave = false, Function = "      +25
            new HeroDef { Id = 8, Name = "   ", DigHours = 24f, PickCost = 60, FirstWave = false, Function = "  +2  " },
        };
        /// &lt;summary&gt;§ +P0-1           &amp;lt; min(60   , 20%×   )          &lt;/summary&gt;
        public static float DigBoostWindowHours(float totalHours)
        {
            float twentyPct = totalHours * 0.2f;
            float cap = 60f / 60f;    // 60   =1
            return System.Math.Min(cap, twentyPct);
        }
        // ——— §       D7/D30/D100·       ———
        public static readonly (int Day, float WoodProd, float WoodDrain, float GrainProd, float GrainDrain, float IronP
        {
            (7,   25920f, 22580f, 12960f, 1300f,  6480f,  2020f),
            (30,  51840f, 55980f, 25920f, 3100f, 17280f,  5150f),
            (100, 93312f, 94380f, 69120f, 7000f, 45360f, 14000f),
        };
        // ——— §    /    ———
        public const int OfflineCapHours = 8;             // §      8h
        public const float AdRescueMinFuel = 240f;         // §  fillTo80Pct minFuel lv1-2
        public const int AdRescueDailyCap = 2;             // §  2/
    }
}
// NightOffline.cs
using System;
using UnityEngine;
namespace Sy.P05.Night
{
    /// &lt;summary&gt;
    ///        § ·8h  ·P03 OfflinePay
    ///                 ≥48h    +          /
    ///               ·             +10%
    ///         ResourceBank/HeroDig/ColdWave ·
    /// &lt;/summary&gt;
    public static class NightOffline
    {
        public const string SeenKey = "p05_night_seen_ms";
        public const int MinClaimMinutes = 60;    //   1
        public static void Stamp(long nowMs)
        {
            PlayerPrefs.SetString(SeenKey, nowMs.ToString());
            PlayerPrefs.Save();
        }
        public static long LastSeen()
        {
            var raw = PlayerPrefs.GetString(SeenKey, "");
```

```
            return long.TryParse(raw, out long v) ? v : 0;
        }
        public sealed class Claim
        {
            public long ElapsedMs { get; }
            public bool Claimable =&gt; ElapsedMs &gt;= MinClaimMinutes * 60_000L;
            public bool Consumed { get; private set; }
            Claim(long elapsedMs) { ElapsedMs = elapsedMs; }
            /// &lt;summary&gt;       →         =         &lt;/summary&gt;
            public static Claim Open(long nowMs)
            {
                long seen = LastSeen();
                if (seen &lt;= 0) return new Claim(0);
                return new Claim(nowMs - seen);
            }
            /// &lt;summary&gt;               8h  ·
            ///    ≥48h                 -1=   /   &lt;/summary&gt;
            public int TryCollect(long nowMs, ResourceBank bank, CampBuilder camp, HeroDig heroes, ColdWave cold)
            {
                if (Consumed || !Claimable) return -1;
                float minutes = ResourceBank.OfflineMinutes(ElapsedMs);
                float hours = minutes / 60f;
                var prod = camp.ProductionPerMinute(heroes.GatherMultiplier());
                float penalty = cold.ProductionPenalty();
                bank.ProductionMultiplier = penalty;
                bank.ApplyOffline((long)(minutes * 60_000L),
                    prod.wood, prod.grain, prod.iron, camp.WarehouseLevel);
                heroes.Tick(hours);   //
                cold.TryExempt(ElapsedMs, bank, campLevelFire(camp), camp);
                cold.ClearPenalty();   //
                Stamp(nowMs);
                Consumed = true;
                return (int)minutes;
            }
            static int campLevelFire(CampBuilder camp) =&gt; 1;   //          UI    cfg
        }
        public static void ClearForTests()
        {
            PlayerPrefs.DeleteKey(SeenKey);
        }
    }
}
// NightWatchHub.cs
using UnityEngine;
using UnityEngine.UI;
namespace Sy.P05.Night
{
    /// &lt;summary&gt;
    ///     V1.4          UGUI·  ×  ×      ·CEO   O-20261010-1245
    ///     HUD      ·  6 +   10          +           +
```

```
///      +         Night   ——             =
///    Editor\NightSetup.cs MenuItem        ·
/// &lt;/summary&gt;
public sealed class NightWatchHub : MonoBehaviour
{
    static readonly Color IceBlue = new Color(0.659f, 0.773f, 0.855f);   // #A8C5DA
    static readonly Color InkBlue = new Color(0.118f, 0.227f, 0.373f);  // #1E3A5F
    static readonly Color WarmOrange = new Color(0.957f, 0.529f, 0.204f); // #F38734
    static readonly Color NightBlack = new Color(0.02f, 0.078f, 0.122f);
    static readonly Color PanelDim = new Color(0.05f, 0.12f, 0.20f, 0.92f);
    Canvas _canvas;
    Sprite _square;
    ResourceBank _bank;
    CampBuilder _camp;
    HeroDig _heroes;
    ColdWave _cold;
    Text _woodText, _grainText, _ironText, _coldText, _toastText;
    RectTransform _heroPanel;
    readonly Text[] _buildingLabels = new Text[NightConfig.Buildings.Length];
    readonly Text[] _heroLabels = new Text[NightConfig.Heroes.Length];
    int _lastWood = -1, _lastGrain = -1, _lastIron = -1;
    string _lastCold = "";
    float _tickAccum;
    void Awake()
    {
        Application.targetFrameRate = 30;
        _canvas = FindObjectOfType&lt;Canvas&gt;();
        _square = MakeSquareSprite();
        _bank = ResourceBank.Load();
        _camp = CampBuilder.Load();
        _heroes = HeroDig.Load();
        _cold = ColdWave.Load();
        _bank.ProductionMultiplier = _cold.ProductionPenalty();
        BuildHud();
        BuildColdLine();
        BuildBuildingGrid();
        BuildHeroPanel();
        BuildToastLine();
    }
    void Update()
    {
        //             dt  →
        _tickAccum += Time.unscaledDeltaTime;
        if (_tickAccum &lt; 1f) return;
        float dtMinutes = _tickAccum / 60f;
        _tickAccum = 0f;
        var prod = _camp.ProductionPerMinute(_heroes.GatherMultiplier());
        _bank.Tick(dtMinutes, prod.wood, prod.grain, prod.iron, _camp.WarehouseLevel);
        _heroes.Tick(dtMinutes / 60f);
        SyncHud();
```

```
}
void SyncHud()
{
    int wood = (int)_bank.Wood, grain = (int)_bank.Grain, iron = (int)_bank.Iron;
    if (wood != _lastWood) { _lastWood = wood; _woodText.text = "  " + wood; }
    if (grain != _lastGrain) { _lastGrain = grain; _grainText.text = "  " + grain; }
    if (iron != _lastIron) { _lastIron = iron; _ironText.text = "  " + iron; }
    float need = ColdWave.NightWoodNeed(1, _camp, _heroes);
    string cold = "   " + (_bank.Wood &gt;= need ? "      " + (int)need + " " : "      " + (int)need + " ");
    if (cold != _lastCold) { _lastCold = cold; _coldText.text = cold; }
}
void BuildHud()
{
    var root = _canvas.transform;
    _woodText = MakeText(root, "  0", 46, new Vector2(-330, 880), TextAnchor.UpperLeft, 320, 70);
    _woodText.color = WarmOrange;
    _grainText = MakeText(root, "  0", 46, new Vector2(0, 880), TextAnchor.UpperLeft, 320, 70);
    _grainText.color = IceBlue;
    _ironText = MakeText(root, "  0", 46, new Vector2(330, 880), TextAnchor.UpperLeft, 320, 70);
    _ironText.color = IceBlue;
    //
    MakeButton("    ", new Vector2(0, -800), InkBlue, TryClaimOffline);
}
void BuildColdLine()
{
    _coldText = MakeText(_canvas.transform, "", 42, new Vector2(0, 800), TextAnchor.MiddleCenter, 900, 60);
    _coldText.color = IceBlue;
}
void BuildBuildingGrid()
{
    var root = _canvas.transform;
    for (int i = 0; i &lt; NightConfig.Buildings.Length; i++)
    {
        int id = NightConfig.Buildings[i].Id;
        int row = i / 2, col = i % 2;
        var btn = MakeButton(NightConfig.Buildings[i].Name + " 0",
            new Vector2(-320 + col * 640, 660 - row * 200), InkBlue, () =&gt; TryUpgrade(id));
        _buildingLabels[i] = btn.GetComponentInChildren&lt;Text&gt;();
        _buildingLabels[i].color = NightConfig.Buildings[i].FirstWave ? IceBlue : new Color(0.45f, 0.55f, 0.62f)
    }
    RefreshBuildings();
}
void RefreshBuildings()
{
    for (int i = 0; i &lt; NightConfig.Buildings.Length; i++)
    {
        var def = NightConfig.Buildings[i];
        int lv = _camp.Level(def.Id);
        string state = lv &gt; 0 ? " Lv" + lv : (def.UnlockDay &gt; _camp.Day ? "D" + def.UnlockDay : "  ");
        _buildingLabels[i].text = def.Name + " " + state;
```

```
    }
}
void TryUpgrade(int id)
{
    if (_camp.TryBuildOrUpgrade(id, _bank))
    {
        RefreshBuildings();
        ShowToast(NightConfig.Buildings[id - 1].Name + "   ");
    }
    else
    {
        var def = NightConfig.Buildings[id - 1];
        if (def.UnlockDay &gt; _camp.Day) ShowToast(def.Name + " " + def.UnlockDay + "    ");
        else ShowToast("       " + NightConfig.BuildCostIron(_camp.Level(id) + 1)
                       + "   " + NightConfig.BuildCostGrain(_camp.Level(id) + 1) + " ");
    }
}
void BuildHeroPanel()
{
    MakeButton("  ", new Vector2(0, -660), IceBlue, ToggleHeroPanel);
    _heroPanel = MakeOverlayPanel();
    MakeText(_heroPanel, "   ·     ", 68, new Vector2(0, 800), TextAnchor.MiddleCenter, 900, 80).color = IceBlue
    for (int i = 0; i &lt; NightConfig.Heroes.Length; i++)
    {
        int heroId = NightConfig.Heroes[i].Id;
        int row = i / 2, col = i % 2;
        var card = new GameObject("Hero_" + heroId, typeof(RectTransform), typeof(Image), typeof(Button));
        card.transform.SetParent(_heroPanel, false);
        var rt = (RectTransform)card.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = new Vector2(-330 + col * 660, 560 - row * 240);
        rt.sizeDelta = new Vector2(600, 210);
        var img = card.GetComponent&lt;Image&gt;();
        img.sprite = _square;
        img.color = PanelDim;
        _heroLabels[i] = MakeText(card.transform, "", 42, new Vector2(0, 40), TextAnchor.MiddleCenter, 580, 70);
        _heroLabels[i].color = IceBlue;
        MakeText(card.transform, NightConfig.Heroes[i].Function, 32, new Vector2(0, -55),
            TextAnchor.MiddleCenter, 580, 60).color = new Color(0.6f, 0.72f, 0.8f);
        card.GetComponent&lt;Button&gt;().onClick.AddListener(() =&gt; TryDig(heroId));
    }
    var close = MakeButton("  ", new Vector2(0, -800), IceBlue, ToggleHeroPanel);
    close.transform.SetParent(_heroPanel, false);
    _heroPanel.gameObject.SetActive(false);
    RefreshHeroes();
}
void RefreshHeroes()
{
    for (int i = 0; i &lt; NightConfig.Heroes.Length; i++)
    {
```

```
            var def = NightConfig.Heroes[i];
            switch (_heroes.States[i])
            {
                case HeroDig.HeroState.Locked:
                    _heroLabels[i].text = def.Name + " ·      " + def.PickCost + " ";
                    break;
                case HeroDig.HeroState.Digging:
                    _heroLabels[i].text = def.Name + " ·    " + (int)_heroes.Progress[i] + "/" + def.DigHours + "h";
                    break;
                default:
                    _heroLabels[i].text = def.Name + " ·    ";
                    _heroLabels[i].color = WarmOrange;
                    break;
            }
        }
    }
    void TryDig(int heroId)
    {
        if (_heroes.StartDig(heroId, _bank, _camp))
        {
            ShowToast("    " + Def(heroId).Name);
        }
        else
        {
            var def = Def(heroId);
            if (_camp.DigQueueSize &lt;= _heroes.DiggingCount()) ShowToast("      ");
            else ShowToast("       " + def.PickCost + " ");
        }
        RefreshHeroes();
    }
    static NightConfig.HeroDef Def(int heroId)
    {
        foreach (var h in NightConfig.Heroes)
            if (h.Id == heroId) return h;
        return null;
    }
    void ToggleHeroPanel()
    {
        bool show = !_heroPanel.gameObject.activeSelf;
        if (show) RefreshHeroes();
        _heroPanel.gameObject.SetActive(show);
    }
    void TryClaimOffline()
    {
        long now = System.DateTimeOffset.Now.ToUnixTimeMilliseconds();
        var claim = NightOffline.Claim.Open(now);
        int minutes = claim.TryCollect(now, _bank, _camp, _heroes, _cold);
        if (minutes &gt; 0) ShowToast("   " + (minutes / 60) + "      ");
        else if (claim.Claimable) ShowToast("    ");
        else ShowToast("     1   ");
```

```
        SyncHud();
    }
    void BuildToastLine()
    {
        _toastText = MakeText(_canvas.transform, "", 40, new Vector2(0, -880), TextAnchor.MiddleCenter, 1000, 70);
        _toastText.color = WarmOrange;
    }
    void ShowToast(string message)
    {
        _toastText.text = message;
        CancelInvoke(nameof(ClearToast));
        Invoke(nameof(ClearToast), 1.6f);
    }
    void ClearToast() =&gt; _toastText.text = "";
    // ——— UI        UGUI·    YAML     ———
    Button MakeButton(string text, Vector2 pos, Color color, System.Action onClick)
    {
        var go = new GameObject("Btn_" + text, typeof(RectTransform), typeof(Image), typeof(Button));
        go.transform.SetParent(_canvas.transform, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = pos;
        rt.sizeDelta = new Vector2(580, 120);
        var img = go.GetComponent&lt;Image&gt;();
        img.sprite = _square;
        img.color = color;
        var label = MakeText(go.transform, text, 40, Vector2.zero, TextAnchor.MiddleCenter, 580, 120);
        label.color = color == IceBlue ? InkBlue : IceBlue;
        var btn = go.GetComponent&lt;Button&gt;();
        btn.onClick.AddListener(() =&gt; onClick());
        return btn;
    }
    RectTransform MakeOverlayPanel()
    {
        var go = new GameObject("NightPanel", typeof(RectTransform), typeof(Image));
        go.transform.SetParent(_canvas.transform, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = Vector2.zero;
        rt.anchorMax = Vector2.one;
        rt.offsetMin = rt.offsetMax = Vector2.zero;
        var img = go.GetComponent&lt;Image&gt;();
        img.sprite = _square;
        img.color = NightBlack;
        return rt;
    }
    static Text MakeText(Transform parent, string text, int size, Vector2 pos, TextAnchor anchor, float w, float h)
    {
        var go = new GameObject("Text", typeof(RectTransform), typeof(Text));
        go.transform.SetParent(parent, false);
        var rt = (RectTransform)go.transform;
```

```
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
            rt.anchoredPosition = pos;
            rt.sizeDelta = new Vector2(w, h);
            var t = go.GetComponent&lt;Text&gt;();
            t.font = Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf");
            t.text = text;
            t.fontSize = size;
            t.alignment = anchor;
            t.color = new Color(0.91f, 0.89f, 0.85f);
            return t;
        }
        static Sprite MakeSquareSprite()
        {
            var tex = new Texture2D(8, 8, TextureFormat.RGBA32, false);
            var px = tex.GetPixels32();
            for (int i = 0; i &lt; px.Length; i++) px[i] = new Color32(255, 255, 255, 255);
            tex.SetPixels32(px);
            tex.Apply(false, false);
            return Sprite.Create(tex, new Rect(0, 0, 8, 8), new Vector2(0.5f, 0.5f), 8f);
        }
    }
}
// ResourceBank.cs
using System;
using UnityEngine;
namespace Sy.P05.Night
{
    /// &lt;summary&gt;
    ///       / / ·§     ·  C#             tick
    ///         12 PlayerPrefs=         =     -15%
    ///          +20%
    /// &lt;/summary&gt;
    public sealed class ResourceBank
    {
        public const string SaveKey = "p05_night_resources";
        const string WoodKey = "p05_night_wood";
        const string GrainKey = "p05_night_grain";
        const string IronKey = "p05_night_iron";
        public float Wood { get; private set; }
        public float Grain { get; private set; }
        public float Iron { get; private set; }
        /// &lt;summary&gt;           /       ·   1  &lt;/summary&gt;
        public float ProductionMultiplier { get; set; } = 1f;
        public static ResourceBank Load()
        {
            var b = new ResourceBank
            {
                Wood = PlayerPrefs.GetFloat(WoodKey, 0f),
                Grain = PlayerPrefs.GetFloat(GrainKey, 0f),
                Iron = PlayerPrefs.GetFloat(IronKey, 0f),
```

```
    };
    return b;
}
void Save()
{
    PlayerPrefs.SetFloat(WoodKey, Wood);
    PlayerPrefs.SetFloat(GrainKey, Grain);
    PlayerPrefs.SetFloat(IronKey, Iron);
    PlayerPrefs.Save();
}
public float CapFor(int warehouseLevel)
    =&gt; NightConfig.StorageBase + warehouseLevel * NightConfig.WarehouseCapPerLevel;
/// &lt;summary&gt;    →tick         ×dt               &lt;/summary&gt;
public void Tick(float dtMinutes, float woodPerMin, float grainPerMin, float ironPerMin, int warehouseLevel)
{
    if (dtMinutes &lt;= 0f) return;
    float cap = CapFor(warehouseLevel);
    Wood = Mathf.Min(cap, Wood + woodPerMin * ProductionMultiplier * dtMinutes);
    Grain = Mathf.Min(cap, Grain + grainPerMin * ProductionMultiplier * dtMinutes);
    Iron = Mathf.Min(cap, Iron + ironPerMin * ProductionMultiplier * dtMinutes);
    Save();
}
/// &lt;summary&gt;     /           false     &lt;/summary&gt;
public bool TrySpend(float wood, float grain, float iron)
{
    if (Wood &lt; wood || Grain &lt; grain || Iron &lt; iron) return false;
    Wood -= wood;
    Grain -= grain;
    Iron -= iron;
    Save();
    return true;
}
public void AddWood(float amount)
{
    Wood += amount;
    Save();
}
/// &lt;summary&gt;  /          ·     &lt;/summary&gt;
public void DevSet(float wood, float grain, float iron)
{
    Wood = wood;
    Grain = grain;
    Iron = iron;
    Save();
}
/// &lt;summary&gt;     §  8h  ·    ·               &lt;/summary&gt;
public float ApplyOffline(long elapsedMs, float woodPerMin, float grainPerMin, float ironPerMin, int warehouseLe
{
    float minutes = OfflineMinutes(elapsedMs);
    Tick(minutes, woodPerMin, grainPerMin, ironPerMin, warehouseLevel);
```

```
            return minutes;
        }
        public static float OfflineMinutes(long elapsedMs)
        {
            if (elapsedMs &lt; 0) return 0f;
            const long MsPerMinute = 60_000L;
            long minutes = elapsedMs / MsPerMinute;
            const long Cap = NightConfig.OfflineCapHours * 60L;
            if (minutes &gt; Cap) minutes = Cap;
            return minutes;
        }
        public static void ClearForTests()
        {
            PlayerPrefs.DeleteKey(WoodKey);
            PlayerPrefs.DeleteKey(GrainKey);
            PlayerPrefs.DeleteKey(IronKey);
            PlayerPrefs.DeleteKey(SaveKey);
        }
    }
}
```